实验四　傅立叶变换与频域滤波

一、实验目的与要求

1. 了解图像变换的意义和手段。
2. 熟悉傅立叶变换的基本性质。
3. 熟练掌握 FFT 变换方法及应用。
4. 通过实验了解二维频谱的分布特点。

二、实验原理及知识点

1、应用傅立叶变换进行图像处理

傅立叶变换是线性系统分析的一个有力工具，它能够定量地分析诸如数字化系统、采样点、电子放大器、卷积滤波器、噪音和显示点等的作用。

2、傅立叶（Fourier）变换的定义

对于二维信号，二维 Fourier 变换定义为：

$$F(u,v)=\int_{-\infty}^{+\infty}\int_{-\infty}^{+\infty} f(x,y)e^{-j2\pi(ux+vy)}dxdy$$

逆变换：

$$f(x,y)=\int_{-\infty}^{+\infty}\int_{-\infty}^{+\infty} F(u,v)e^{j2\pi(ux+vy)}dudv$$

二维离散傅立叶变换为：

$$F(u,v)=\sum_{x=0}^{M-1}\sum_{y=0}^{N-1} fF(x,y)e^{-j2\pi(\frac{ux}{M}+\frac{vy}{N})}$$

逆变换：

$$f(x,y)=\frac{1}{MN}\sum_{u=0}^{M-1}\sum_{v=0}^{N-1} F(u,v)e^{j2\pi(\frac{ux}{M}+\frac{vy}{N})}$$

下面给出傅立叶变换/频谱平移/傅立叶逆变换的 MATLAB 和 Python 函数:

Matlab: output=fft2(input) / f_shift=fftshift(f) / f= ifftshift(f_shift) / output=ifft2(input)

Python: output= np.fft.fft2(input) / f_shift=np.fft.fftshift(f) /

f= np.fft.ifftshift(f_shift) / output= np.fft.ifft2(input)

2、频域滤波

数字图像平移后的频谱中，图像的能量将集中到频谱中心（低频成分），图像上的边缘、线条细节信息（高频成分）将分散在图像频谱的边缘。也就是说，频谱中低频成分代表了图像的概貌，即灰度变换平缓的部分；高频成分代表了图像中的细节，即图像中的边缘或噪声。因此，合理的选择滤波器，通过在频域中对相应频率成分进行抑制或增强，便可完成图像的增强处理。

在频域中进行图像处理的步骤如下：

(1)计算图像的 DFT，得到$F(u,v)$；

(2)用滤波函数$H(u,v)$乘以$F(u,v)$，得到处理结果$G(u,v)$；

(3)计算滤波后的 IDFT；

(4)取 IDFT 变换结果中的实部，得到处理后的图像。

$H(u,v)$称作滤波器，它具有允许某些频率成分通过，而阻止其他频率成分通过的特性。该处理过程可以表示为：

$$G(u,v) = F(u,v)H(u,v)$$

频域滤波器的基本类型有：低通滤波器、高通滤波器、带通滤波器和带阻滤波器。

2.1、低通滤波器

低通滤波器允许低频成分通过，而抑制高频成分。因此，它能够去除图像中的噪声，实现图像平滑操作。当然，这必然会引起图像模糊。理想低通滤波器的滤波函数为：

$$H(u,v) = \begin{cases} 1, & D(u,v) \le D_0 \\ 0, & D(u,v) > D_0 \end{cases}$$

2.2、高通滤波器

与低通滤波器相反，高通滤波器则允许高频成分通过，而抑制低频成分。因此，它能够强化图像中目标的边缘，起锐化作用。但它同时也强化了图像中的噪声。理想高通滤波器的滤波函数为：

$$H(u,v) = \begin{cases} 0, & D(u,v) \le D_0 \\ 1, & D(u,v) > D_0 \end{cases}$$

2.3、巴特沃斯滤波器

由于理想滤波器存在明显的振铃现象，且其垂直的频率响应特性仅能用软件方法实现，无法用电路实现。因此，研究实用滤波器极具应用价值。一种常用的频域滤波器是巴特沃斯滤波器。

低通巴特沃斯滤波器的滤波函数为：

$$H(u,v) = \frac{1}{1 + [D(u,v)/D_0]^{2n}}$$

高通巴特沃斯滤波器的滤波函数为：

$$H(u,v) = \frac{1}{1 + [D_0/D(u,v)]^{2n}}$$

式中：$D(u,v) = \sqrt{(u^2 + v^2)}$；n 为巴特沃斯滤波器阶数，$D_0$为指定的非负值，称为截止频率。

3、实验完整代码

Matlab 代码：

```
clc;clear;close all;
grayimg=imread('testimg/camera.png');
grayimg=im2double(grayimg);        %  图像数据类型转换

f=fft2(grayimg);                   % FFT  傅里叶变换  f 是一个复数
f_shift=fftshift(f);               % shift  将变换的频率图像四角移动到中心

subplot(3,3,1),imshow(grayimg),title('original img');
subplot(3,3,2),imshow(log(abs(f)+1),[]),title('FFT before shift');
%  加对数是为了更好的显示
subplot(3,3,3),imshow(log(abs(f_shift)+1),[]),title('FFT after shift');
% []会根据灰度图的数值范围来显示图像

[m,n]=size(f_shift);
m_mid=fix(m/2);
n_mid=fix(n/2);

l=zeros(m,n);                      %  理想低通滤波
d0=50;
for i=1:m
    for j=1:n
        d=sqrt((i-m_mid)^2+(j-n_mid)^2);
        if d<=d0
            l(i,j)=1.0;
        end
    end
end
img_lpf=l.*f_shift;                %  滤波操作
img_lpf=ifftshift(img_lpf);        %  逆 shift 操作
img_lpf=ifft2(img_lpf);            %  反傅里叶变换  输出一个复数
img_lpf=real(img_lpf);             %  取复数单元的实数部分构成灰度图像
subplot(3,3,4);imshow(l);title('low pass filter');
subplot(3,3,7);imshow(img_lpf);title('low pass filterd image');
```

```
l2=zeros(m,n);                          % 巴特沃兹低通滤波
d2=50;
N=2;
for i=1:m
    for j=1:n
        d=sqrt((i-m_mid)^2+(j-n_mid)^2);
        lpf=1/(1+(d/d2)^(2*N));
        l2(i,j)=lpf;
    end
end
img_blpf=l2.*f_shift;
img_blpf=ifftshift(img_blpf);
img_blpf=ifft2(img_blpf);
img_blpf=real(img_blpf);
subplot(3,3,6);imshow(l2);title('BLPF filter');
subplot(3,3,9);imshow(img_blpf);title('BLPF filterd image');
```

Python 代码：

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import math

def main():
    grayimg = cv2.imread('testimg/camera.png', cv2.IMREAD_GRAYSCALE)
    plt.subplot(331), plt.imshow(grayimg, cmap='gray'), plt.title('original img'), plt.axis('off')

    grayimg = np.float64(grayimg/255.0)
    f = np.fft.fft2(grayimg)

    magnitude_spectrum1 = np.log(np.abs(f)+1)
    plt.subplot(332), plt.imshow(magnitude_spectrum1, cmap='gray'), plt.title('FFT before shift'),
plt.axis('off')

    fshift = np.fft.fftshift(f)
```

```
magnitude_spectrum2 = np.log(np.abs(fshift) + 1)
plt.subplot(333), plt.imshow(magnitude_spectrum2, cmap='gray'), plt.title('FFT after shift'), plt.axis('off')

rows, cols = grayimg.shape
crow, ccol = int(rows / 2), int(cols / 2)  # 找到中心位置

# 理想低通滤波掩膜
D0 = 50
mask = np.zeros((rows, cols), np.float32)
for i in range(rows):
    for j in range(cols):
        if (i - crow) ** 2 + (j - ccol) ** 2 <= D0 ** 2:
            mask[i, j] = 1.0

# 理想高通滤波掩膜
D1 = 50
mask2 = np.ones((rows, cols), np.float32)
for i in range(rows):
    for j in range(cols):
        if (i - crow) ** 2 + (j - ccol) ** 2 <= D1 ** 2:
            mask2[i, j] = 0.0

m_fshift = fshift * mask
m_fshift2 = fshift * mask2

ishift = np.fft.ifftshift(m_fshift)
ishift2 = np.fft.ifftshift(m_fshift2)

iimg = np.fft.ifft2(ishift)
iimg2 = np.fft.ifft2(ishift2)

iimg = np.abs(iimg)
iimg2 = np.abs(iimg2)
```

```
    plt.subplot(334), plt.imshow(mask, cmap='gray'), plt.title('low pass filter'), plt.axis('off')
    plt.subplot(337), plt.imshow(iimg, cmap='gray'), plt.title('low pass filterd image'), plt.axis('off')
    plt.subplot(335), plt.imshow(mask2, cmap='gray'), plt.title('high pass filter'), plt.axis('off')
    plt.subplot(338), plt.imshow(iimg2, cmap='gray'), plt.title('high pass filterd image'), plt.axis('off')
    plt.show()

if __name__ == '__main__':
    main()
```

三、实验内容及步骤

Matlab 部分：

1．使用 imread()函数读取一幅灰度图像并用 im2double()转换数据类型；
2．使用 fft2 ()函数对图像进行傅立叶变换；
3．使用 fftshift ()函数进行频谱平移
4．设计滤波器，将其与频谱图像相乘实现低通/高通滤波；
5．使用 ifftshift ()函数进行逆频谱平移；
6．使用 ifft2 ()函数进行逆傅立叶变换；
7．使用 real ()函数取逆变换后复数的实数部分构成灰度图像；
8．使用 imshow ()函数显示原始图像/平移前后频谱图像/滤波器图像/滤波后图像；
9．观察比较不同滤波器对图像的处理效果；
10．更改滤波器的截止频率大小，重复第 4~8 步，观察效果；
11．将每一步的函数执行语句拷贝下来，写入实验报告，并将得到的图像结果保存。

Python 部分：

1．使用 cv2.imread()函数读取一幅灰度图像并用 np.float64 ()转换数据类型；
2．使用 np.fft.fft2 ()函数对图像进行傅立叶变换；
3．使用 np.fft.fftshift ()函数进行频谱平移
4．设计滤波器，将其与频谱图像相乘实现低通/高通滤波；
5．使用 np.fft.ifftshift ()函数进行逆频谱平移；
6．使用 np.fft.ifft2 ()函数进行逆傅立叶变换；
7．使用 np.abs ()函数取逆变换后复数的模构成灰度图像；
8．使用 plt.imshow ()函数显示原始图像/平移前后频谱图像/滤波器图像/滤波后图像；
9．观察比较不同滤波器对图像的处理效果；
10．更改滤波器的截止频率大小，重复第 4~8 步，观察效果；
11．将每一步的函数执行语句拷贝下来，写入实验报告，并将得到的图像结果保存。

四、考核要点

1、 了解在 Matlab / Python 中如何对图像进行傅立叶变换、傅立叶逆变换。

2、 掌握在 Matlab / Python 中如何对频域图像进行低通滤波和高通滤波。

3、 观察不同滤波器、不同截止频率对图像滤波效果的影响。

4、 仿照已经给出的代码，补全 Matlab 中的理想高通滤波和 Python 中的巴特沃斯低通滤波代码，显示掩膜和滤波后图像。

五、实验仪器与软件

(1) PC 计算机

(2) MatLab 软件/语言包括图像处理工具箱(Image Processing Toolbox) ;
Python 语言环境/编辑器/ OpenCV 扩展包(cv2)/ matplotlib 扩展包/math 扩展包

(3) 实验所需要的测试图片

六、实验报告要求

描述实验的基本步骤，用数据和图片给出各个步骤中取得的实验结果和源代码，并进行必要的讨论，必须包括原始图像及其计算/处理后的图像。

七、思考题

(1) 傅立叶变换有哪些重要的性质？

(2) 图像的二维频谱在显示和处理时应注意什么？

(3) 比较理想低通滤波和巴特沃斯低通滤波后图像的差异，分析并解释其原因。

八、实验图像

Fig.4 camera.png